

Vulnrichment Viewer 仕様書

Version 1.0 / 2026年7月

公開URL	https://cyberdivemaster.github.io/vulnviewer/
リポジトリ	https://github.com/CyberDiveMaster/vulnviewer
データソース	CISA Vulnrichment (https://github.com/cisagov/vulnrichment)
ホスティング	GitHub Pages (静的配信、サーバーレス)

1. 概要

Vulnrichment Viewerは、CISAが公開している脆弱性エンリッチメントデータセット

「Vulnrichment」を、CVSS値・SSVC評価（Exploitation / Automatable / Technical Impact）・CVSSベクタ成分（AV/AC/PR/UI）・Vendor / Product / CWEなど多様な条件でフィルタ・ソートしながら閲覧できる軽量Webビューワーである。

既存の類似ツールを探したが見当たらなかったため、個人で開発した。バックエンドサーバーを持たず、GitHub ActionsによるデータのバッチクロールとGitHub Pagesによる静的配信のみで構成されており、運用コストは実質ゼロである。

2. 背景・目的

- Vulnrichmentが提供するSSVC評価値（Exploitation等）は現在値のスナップショットのみで、「いつActiveになったか」という履歴情報を直接は提供していない。本ツールはVulnrichmentリポジトリのgit履歴を独自にマイニングし、CVEの公開日からSSVC Exploitation評価がActiveになるまでの日数を算出・表示する点が最大の特徴である。
- サーバー運用・DBホスティングのコストと管理負担を避けるため、GitHub Actions + GitHub Pages + クライアントサイドJavaScriptのみで完結する構成を採用した。

3. システム構成

3.1 全体アーキテクチャ

- 初回バックフィル**（ローカル・手動、1回のみ）：Vulnrichmentリポジトリの全git履歴をマイニングし、SQLite DB（`data/vulnviewer.db`）を構築する。

2. **定期更新** (GitHub Actions、1日2回・cron) : 前回処理分からの新規コミットのみを取り込み、DBを差分更新する。DB自体はサイズが大きい(約180MB超)ためGitにはコミットせず、GitHub Actionsのキャッシュ機能のみで永続化する。
3. **静的JSON書き出し**: DBの内容をフラットな1CVE=1行のJSONに変換し、gzip圧縮した上で `docs/data/cves.json.gz` としてリポジトリにコミットする (GitHub Pagesがそのまま配信)。
4. **フロントエンド**: ブラウザがgzip済みJSONを取得・解凍し、Tabulator.js (テーブル表示・フィルタ・ソート) とFlatpickr (日付レンジ選択カレンダー) を用いて全件クライアントサイドで表示する。バックエンドAPIは存在しない。

3.2 技術スタック

レイヤー	技術	役割
データ取得・加工	Python 3 (標準ライブラリのみ、git CLIを subprocess経由で使用)	Vulnrichmentのクローン・履歴マイニング・SQLite管理・JSON書き出し
永続化	SQLite	CVEスナップショット・Exploitation状態遷移履歴の保管
自動化	GitHub Actions (cron: 毎日 UTC 0時・12時)	データの定期取得・更新・コミット
配信	GitHub Pages (main ブランチ /docs)	静的ファイルのホスティング
UI	Tabulator.js 6.3.0 / Flatpickr 4.6.13 (いずれもCDN経由、ビルド不要)	テーブル表示・フィルタ・ソート・日付レンジ選択

4. データソースとフィールドマッピング

Vulnrichmentは1CVEにつき1つのJSONファイル (パス例: `2025/32xxx/CVE-2025-32432.json`、CVE 5.xフォーマット) を持つ。主なマッピングは以下の通り。

Vulnrichment側の項目	取得元パス
Exploitation / Automatable / Technical Impact	<code>containers.adp[].metrics[].other.content.options[]</code> (SSVC評価、CISA ADP)
CVSSベクタ・スコア	<code>containers.cna.metrics[].cvssV3_1</code> 等 (CNA提供)、ADP提供分も別途取得
Vendor / Product	<code>containers.cna.affected[]</code>
CWE	<code>containers.cna.problemTypes[].descriptions[]</code>
公開日・予約日・更新日	<code>cveMetadata.datePublished / dateReserved / dateUpdated</code>

5. データベーススキーマ (SQLite)

テーブル	役割
meta	処理済みgitコミットSHA・最終更新日時などの状態管理
cve	CVEごとの現在スナップショット (CVSS・SSVC評価値・派生列を含む)
cve_vendor_product	Vendor/Productの1:N関係
cve_cwe	CWEの1:N関係
cve_cvss	CNA/ADP双方のCVSSベクタ (1CVEに複数存在しうる)
exploitation_history	Exploitation値の状態遷移履歴 (git履歴マイニングにより構築、日数計算の根拠データ)

5.1 派生列 (cve テーブル)

列名	説明
first_none_date / first_poc_date / first_active_date	各Exploitation値が最初に観測された日時
exploitation_left_censored	git履歴上の初出時点で既にpoc/activeだった (遷移元が観測不能) ことを示すフラグ
days_publish_to_active	CVE公開日からExploitationが最初にActiveになるまでの日数。公開当日にActive化した場合は0。Active化した記録が一度もない場合はNULL

Exploitation状態の履歴は、Vulnrichmentリポジトリのgit履歴を `git log -G "Exploitation"` によるピックアップ検索で絞り込み、コミット単位でJSONを構造的に比較することで検出している (フォーマット変更のみのコミットを誤検知しないための対策)。

6. フロントエンド仕様

6.1 画面構成

- ヘッダー: タイトル、データ件数・最終更新時刻
- ツールバー: フィルタ該当件数表示、CSVエクスポートボタン
- テーブル本体: 全件クライアントサイドでロードし、Tabulatorの仮想DOMで表示 (ページネーションなし)

6.2 列とフィルタ仕様

列	フィルタ方式	備考
CVE ID	テキスト部分一致	cve.org へのリンク付き
Date Published	日付レンジ（Flatpickrカレンダー、From/To）	
Active Since	日付レンジ（Flatpickrカレンダー、From/To）	<code>first_active_date</code>
Exploitation	選択式（none / poc / active / 評価なし）	
Automatable	選択式（yes / no / 評価なし）	
Technical Impact	選択式（partial / total / 評価なし）	
CVSS Score	最小値入力	入力値以上のスコアを表示
AV / AC / PR / UI	選択式	主CVSSベクタからクライアントサイドで抽出（CVSS v3.x / v4.0双方の値域に対応）
Vendor / Product / CWE	テキスト部分一致	
Days: Publish→Active	ソートのみ	NULLは「N/A」表示
Last Updated	ソートのみ	<code>date_updated</code>

初期表示は **Active Since の新しい順（降順）**。全列で複数列ソート（Shiftクリック）に対応。

6.3 CSVエクスポート

- 「Export CSV」ボタン押下時、**現在適用中の全フィルタを通過した行のみ**（表示中のソート順）をCSV化する。
- 該当件数が5,000件を超える場合はエクスポートを行わず、件数を絞り込むよう促すエラーメッセージを表示する（スプレッドシートでの実用性を考慮した上限）。

6.4 データ鮮度

- JSONデータ取得は `cache: "no-cache"` を指定し、ページ読み込み毎に必ずサーバーへ再検証を行う（GitHub PagesのCDNキャッシュにより古いデータが表示され続けることを防止）。
- ページ自体（HTML/JS）はブラウザ・CDNキャッシュの影響を受けるため、コード変更直後はハードリロード（Ctrl+Shift+R）が必要な場合がある。

7. 運用・制限事項

- データ更新頻度: 1日2回（UTC 0時・12時）。GitHub Actions（Publicリポジトリのため実行時間無制限）で自動実行。
- SQLite DBはGitHub Actionsのキャッシュのみで永続化しており、Gitへはコミットしていない（ファイルサイズがGitHubの1ファイル100MB制限を超えるため）。キャッシュが失効した場合は、次回実行時に全履歴マイニングを自動的にやり直す（データの整合性は保たれるが、その回のみ処理時間が延びる）。
- Exploitation状態の遷移日数（Publish→Active等）は、Vulnrichmentのgit履歴上で実際に観測できた範囲でのみ算出される。履歴上の初出時点で既にActive/PoCだったCVEについては遷移元が不明なため、該当項目は「N/A」として扱う。
- SSVC評価自体が存在しないCVE（CNAのみのレコード）は、Exploitation等の値がNULL（未評価）となり、文字列としての"none"とは区別される。